System Architecture

1. Purpose

This document explains how the DyslexiaDetectionSystem is assembled and how the pieces interact at runtime. It is meant to answer:

- what each component does
- how data moves through the system
- how the model families differ
- how the dashboards connect to the core package
- how records, reports, and deployment artifacts are produced

The project is multimodal and multi-surface. That means one feature may exist in more than one UI, but the backend logic and persistence layer can still be different. When debugging, always identify the active surface first. The documentation frames the system as a learning-disorder platform, with dyslexia as the primary use case.

2. High-Level View

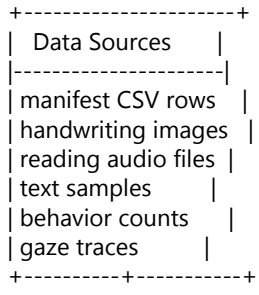
The repository has four primary layers:

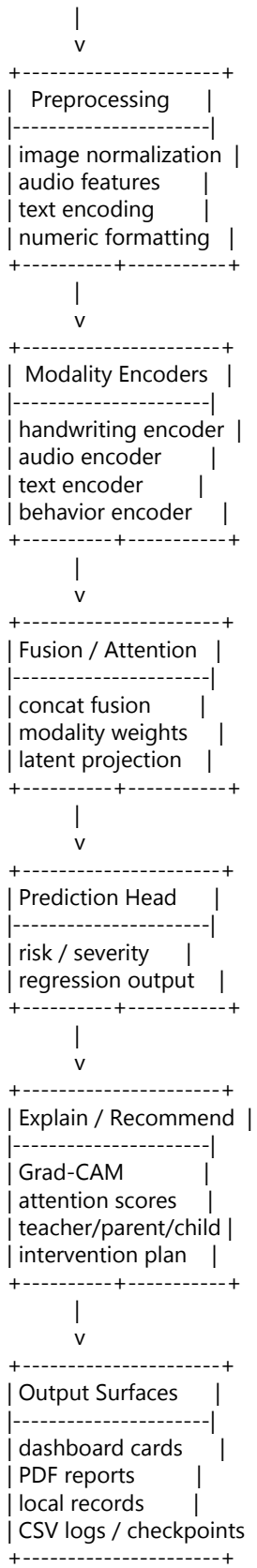
1. Data and collection layer
1. Preprocessing and model layer
1. Explanation and recommendation layer
1. User interface and deployment layer

In simplified form:

```
Input data
  -> preprocessing
  -> modality encoders
  -> fusion / attention
  -> prediction
  -> explanation / recommendations
  -> report / records / export
```

3. Top-Level Architecture





4. Data and Manifest Layer

4.1 Core manifest contract

Most training and analysis flows begin from a CSV manifest. The shared schema is defined in src/dyslexiadetection/schemas.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/schemas.py).

Required fields include:

- sample_id
- student_hash
- handwriting_path
- audio_path
- text_sample
- spelling_errors
- pronunciation_errors
- label

Behavior-related fields include:

- readingtimeseconds
- hesitation_count
- repetition_count
- omission_count

Eye-tracking fields are tracked separately in the eye collection pipeline.

4.2 Why the manifest is central

The manifest is the single source of truth for:

- dataset loading
- sample lookup
- multimodal training
- severity derivation
- biomarker generation
- local report generation

The system uses path resolution relative to the manifest location so the dataset can stay portable.

5. Preprocessing Layer

The preprocessing logic is implemented in src/dyslexiadetection/preprocessing.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/preprocessing.py).

5.1 Handwriting preprocessing

Working principle:

- load the image in grayscale
- normalize contrast
- pad to a fixed square canvas
- convert to a single-channel float tensor

Why it matters:

- the handwriting model expects a fixed spatial size
- grayscale keeps the representation compact
- contrast normalization makes pen strokes easier to learn

5.2 Audio preprocessing

Working principle:

- read mono WAV data
- resample if needed
- compute a log-spectrogram-like representation
- normalize the final feature map
- pad or crop to a fixed number of frames

Why it matters:

- audio lengths vary in real use
- the network needs stable tensor shapes
- spectral representation highlights pauses, energy, and timing cues

5.3 Text preprocessing

Working principle:

- normalize Unicode to NFC
- lowercase English/Latin text
- tokenize at the character level
- encode into integer IDs
- pad to a fixed maximum length

Why it matters:

- the project needs to support Bengali and multilingual scripts

• character-level tokenization avoids word-vocabulary brittleness in low-resource settings
• fixed length makes batch training simpler

5.4 Behavior feature handling

Behavior features are numeric values such as:

• reading time
• hesitations
• repetitions
• omissions

These are passed as a compact vector so the model can learn how fluency-related behavior interacts with other modalities.

6. Model Layer

The model families are defined in src/dyslexiadetection/models.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py).

6.1 Encoder pattern

Most models use the same conceptual pattern:

1. one encoder per modality
1. one fusion step
1. one prediction head

The encoders are intentionally shallow enough to run locally, but expressive enough to combine multiple educational signals.

6.1.1 Evaluation discipline

The evaluation flow is intentionally split into:

1. cross-validation for model selection
1. a hard holdout split for sanity-checking the selected model
1. ranking and matrix views for the dashboard and docs

That separation helps keep model comparison more trustworthy than a single accuracy number.

6.2 Handwriting encoder

The handwriting encoder is a small CNN stack:

- Conv2d
- BatchNorm
- ReLU
- MaxPool
- repeated feature extraction
- adaptive pooling
- linear projection

It produces a compact handwriting embedding.

6.3 Audio encoder

The audio encoder is a 1D convolution stack over spectral frames.

It is designed to learn:

- energy changes
- timing structure
- local spectral patterns

6.4 Text encoders

The repository contains multiple text encoders:

- TextEncoder using a bidirectional GRU
- LSTMTextEncoder using a bidirectional LSTM
- TransformerTextEncoder using learned positional embeddings and transformer blocks

Working principle:

- all three convert token sequences into a fixed-size sentence embedding
- the transformer version is used when sequence context is especially important

6.5 Vision Transformer handwriting encoder

ViTHandwritingEncoder uses patch embeddings instead of a pure CNN.

Working principle:

- split image into patches
- embed each patch

- prepend a class token
- add positional embeddings
- pass through transformer encoder layers

This is helpful when patch-level structure or global layout matters.

6.6 Behavior encoder

BehaviorEncoder is a small MLP.

Its job is to transform numeric reading-behavior signals into a learned embedding that can be fused with the other modalities.

6.7 Fusion classifiers

There are two main fusion styles:

Concatenation fusion

Used by FusionClassifier and the base multimodal models.

Working principle:

- encode each modality
- concatenate embeddings and error features
- run through dense layers
- output logits or regression values

Attention fusion

Used by AttentionFusionClassifier and AttentionMultimodalModel.

Working principle:

- project each modality to a shared dimension
- compute a learned attention score
- normalize scores with softmax
- build a weighted sum of the modality embeddings

This lets the model expose modality importance for interpretation.

7. Model Families

The current supervised comparison trio is:

- AttentionMultimodalModel
- TransformerMultimodalModel
- ViTMultimodalModel

Legacy baselines remain available for historical comparison and experimentation.

7.1 Initial baselines

The repository includes:

- InitialCNNModel
- InitialLSTMModel
- InitialCNNLSTMModel

Purpose:

- provide compact baselines
- isolate image, text, and combined multimodal behavior
- preserve historical comparison with the older five-model era

7.2 Default multimodal model

MultimodalDyslexiaModel is the general-purpose screening model.

It combines:

- handwriting CNN
- audio encoder
- GRU text encoder
- behavior encoder
- error counts

This is the standard multimodal screening path.

7.3 Transformer multimodal model

TransformerMultimodalModel swaps the text branch for a transformer encoder.

Use this when sequence context matters more than the lighter GRU branch.

7.4 ViT multimodal model

ViTMultimodalModel swaps the handwriting CNN for a patch-based vision transformer encoder.

Use this when you want patch-aware handwriting representation.

7.5 ViT + Transformer multimodal model

ViTTransformerMultimodalModel uses transformer encoders for both handwriting and text.

This is the most transformer-heavy supervised multimodal option in the repository.

7.6 Attention multimodal model

AttentionMultimodalModel adds explicit modality attention.

This model is useful when you want to inspect which modality contributed most to a prediction.

8. Training Architecture

The main training entry point is src/dyslexiadetection/train.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/train.py).

8.1 Training flow

1. Load manifest through DyslexiaManifestDataset
1. Split into training and validation subsets
1. Build the requested model family
1. Optionally initialize from a pretrained audio encoder
1. Optionally transfer matching weights from a source checkpoint
1. Optionally distill features from a teacher model
1. Train with AdamW
1. Evaluate on validation data each epoch
1. Save best checkpoint and training history

8.2 Task modes

Supported task types:

- binary
- severity
- regression

Meaning:

- binary predicts low risk vs elevated risk
- severity predicts mild / moderate / severe
- regression predicts a continuous severity score

8.3 Why this structure is used

The training loop is intentionally generic so that:

- the same dataset can support multiple tasks
- the same model family can be repurposed
- the same manifest can drive baseline, severity, and transfer-learning experiments

9. Foundation and Transfer Architecture

9.1 Foundation model

src/dyslexiadetection/foundation.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/foundation.py) contains the multimodal foundation model.

It is designed to learn reusable representations before task-specific adaptation.

Core idea:

- encode handwriting, audio, text, and behavior
- project everything into a common latent space
- apply normalization
- optimize with a mixture of contrastive, masked-text, and reconstruction losses

9.2 Adapter head

The adapter adds a small task-specific head for:

- dyslexia
- dysgraphia
- dyscalculia

Working principle:

- freeze or reuse the shared foundation encoder
- train a lighter head for the target disorder

• keep the representation reusable across related tasks

9.3 Cross-lingual transfer

src/dyslexiadetection/crosslingual.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/crosslingual.py) supports weight transfer across source and target language models.

Working principle:

• copy matching tensors by name and shape
• limit transfer to desired module prefixes
• optionally freeze transferred branches
• optionally add feature-level distillation

This is useful for English-to-Bengali or other low-resource transfer scenarios.

9.4 SSL pretraining

src/dyslexiadetection/sslpretraining.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/sslpretraining.py) supports self-supervised audio pretraining.

Working principle:

• create augmented audio views
• learn invariant embeddings
• optionally reconstruct masked regions
• optionally distill from a teacher model

The downstream benefit is a better initialized audio encoder for supervised multimodal training.

10. Explainability Architecture

The explainability layer is intentionally layered so different model families can be interpreted in different ways.

10.1 Grad-CAM

Implemented in src/dyslexiadetection/explainability.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/explainability.py).

Use:

• visualizing what parts of a handwriting image influenced the prediction

10.2 Vision transformer attention heatmap

Use:

• patch importance visualization for ViT handwriting models

10.3 Text attention scores

Use:

• token-level introspection for transformer text models

10.4 Educational explanations

Implemented in src/dyslexiadetection/educationalexplanations.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/educationalexplanations.py).

It turns raw results into:

• teacher-facing advice
• parent-facing advice
• student-facing encouragement
• next steps

This is what makes the system educationally usable rather than purely predictive.

11. Intervention and Therapy Architecture

11.1 Personalized intervention

src/dyslexiadetection/intervention.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/intervention.py) selects exercises based on the current learner profile.

It uses:

• severity level
• spelling and pronunciation errors
• reading time
• hesitations, repetitions, omissions

to select:

• reading exercise
• pronunciation exercise
• spelling exercise
• weekly target minutes

11.2 Adaptive tutoring

src/dyslexiadetection/adaptivetutoring.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/adaptivetutoring.py) implements a lightweight RL-style tutor policy.

Working principle:

• convert learner performance into a discrete state
• select an action using Q-values and exploration
• update the policy with reward from improvement

11.3 Speech therapy

src/dyslexiadetection/speechtherapy.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/speechtherapy.py) contains language-specific therapy tasks and scoring.

Working principle:

• choose a therapy task based on language and level
• record session metadata
• score the session from duration and error counts
• persist the session to CSV

12. Biomarker Architecture

src/dyslexiadetection/biomarkers.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/biomarkers.py) builds a biomarker dataset from the manifest.

12.1 Feature families

• handwriting biomarkers
• speech biomarkers
• reading biomarkers

12.2 Ranking logic

The discovery step ranks biomarkers using:

- Cohen's d
- label correlation
- logistic regression coefficient magnitude

This creates a practical importance ranking rather than relying on only one statistic.

13. Eye-Tracking Architecture

src/dyslexiadetection/eyetracking.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/eyetracking.py) computes gaze-based metrics from a trace table.

Metrics include:

- fixation duration
- regressions
- reading speed
- gaze dispersion
- scanpath length
- mean saccade velocity

These are written into a CSV log for later analysis.

14. Deployment and Optimization Architecture

14.1 Pruning

src/dyslexiadetection/optimization.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/optimization.py) can prune weights from convolutional and linear layers.

Goal:

- smaller models
- fewer active parameters

14.2 Quantization

Dynamic quantization is used for CPU-friendly inference.

Goal:

• smaller artifact size
• faster inference on local hardware

14.3 TorchScript export

TorchScript export converts a trained model into a portable deployment artifact.

Goal:

• easier loading outside a Python training session
• deterministic local runtime artifact

14.4 Federated training

src/dyslexiadetection/federated.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/federated.py) supports local-client training and global weight aggregation.

Goal:

• preserve locality of data
• still learn a shared global model

15. Dashboard Architecture

15.1 Streamlit dashboard

app.py (/d:/Project/DyslexiaDetectionSystem/app.py) is the most complete operational dashboard.

It contains:

• sample collection
• screening
• live webcam analysis
• therapy
• eye tracking
• biomarkers
• final report
• federated and deployment utilities

15.2 Standalone local web dashboard

web/index.html (/d:/Project/DyslexiaDetectionSystem/web/index.html), web/app.js (/d:/Project/DyslexiaDetectionSystem/web/app.js), and web/styles.css (/d:/Project/DyslexiaDetectionSystem/web/styles.css) build the browser-first interface.

Runtime behavior:

- local HTTP server from runlocalweb.py
- browser-based screening workflow
- local storage-backed records
- PDF export
- microphone access on secure localhost origin

15.3 React frontend

frontend/src/App.jsx (/d:/Project/DyslexiaDetectionSystem/frontend/src/App.jsx) contains the API-connected React UI.

It is a separate frontend path and should be maintained independently from web/.

16. End-to-End Runtime Flow

16.1 Typical screening path

```
manifest row
  -> preprocessing
  -> multimodal model
  -> logits / probabilities
  -> explanation text
  -> intervention suggestion
  -> report / record
```

16.2 Typical browser-dashboard path

```
user input
  -> JS scoring / local capture
  -> local record storage
  -> report builder
  -> PDF export
```

16.3 Typical Streamlit path

```
uploaded files / manifest
  -> Python pipeline
  -> model inference
  -> visual analytics
  -> explanations
  -> report generation
```

17. Persistence Model

Different surfaces persist data differently.

- Dataset and training flows use CSV manifests
- Therapy, tutoring, and eye-tracking flows use CSV logs
- Training uses checkpoint directories
- The standalone web dashboard stores records locally in the browser
- Reports are exported as PDFs or saved analysis files

This matters because the same feature may be stored in a different place depending on the active surface.

18. Working Principles That Guide The Codebase

18.1 Multimodal robustness

The system intentionally combines several weak or partial signals. That makes it less dependent on any single observation.

18.2 Language sensitivity

Bengali, English, and multilingual flows are all supported in text handling and the UI.

18.3 Educational output

The system should explain itself in terms a teacher, parent, or learner can understand.

18.4 Local-first deployment

Many workflows are designed to run on a local machine, with or without a browser backend.

18.5 Screening, not diagnosis

Results should be treated as support material and reviewed with a professional or educator.

19. Practical Debugging Rules

1. Identify the active surface first.
1. Trace the exact feature in that surface.
1. Check whether the same feature also exists in another surface.
1. Verify which persistence layer is being used.
1. Update the docs whenever you rename a tab, model, manifest field, or report field.

20. Recommended Reading Order

If you want to understand the architecture quickly, read these files in order:

1. src/dyslexiadetection/config.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/config.py)
1. src/dyslexiadetection/schemas.py
(/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/schemas.py)
1. src/dyslexiadetection/preprocessing.py
(/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/preprocessing.py)
1. src/dyslexiadetection/models.py
(/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py)
1. src/dyslexiadetection/train.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/train.py)
1. src/dyslexiadetection/architecture.py
(/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/architecture.py)
1. web/app.js (/d:/Project/DyslexiaDetectionSystem/web/app.js)
1. app.py (/d:/Project/DyslexiaDetectionSystem/app.py)